Exercise Set 2

Prof. Kenny Paterson
K. T. Truong, S.-P. Merz, Y. Song

In this exercise sheet, we will introduce the Schnorr signature scheme, which has been used in Bitcoin since the Taproot update in 2021 due to its nice algebraic properties. We will then show that one can perform a key recovery attack on this signature scheme if information is given about the nonces used in the signing process. We will first show some simple attacks based on known and repeated nonces. We will then cast the problem of finding the private key given the most significant bits of a few nonces as an instance of the Hidden Number Problem (HNP), which we will reduce to an instance of the Closest Vector Problem (CVP), seen in the lectures. This is one of the most important applications of lattices in cryptanalysis, and is a good example of how to use the tools from the lectures to attack a real-world cryptographic scheme.

Focus during the exercise session: During the exercise session, we will not cover Problem 1, except for the description of the Schnorr signature scheme in Part 1.c. We suggest you work through Problem 1 on your own, so you understand the logic behind the scheme, without having to take the formulae for granted.

For Problem 2, we will focus on the intuition needed to understand the attack, specifically Parts 2.a, 2.c, 2.d, 2.f and 2.g. We will mostly skip the computations, and we strongly suggest you work through them on your own.

Problem 1. Throughout this entire exercise sheet, let $\mathbb{G}$ be a cyclic group of prime order q with generator P, in which computing discrete logarithms is assumed to be hard. We will be working over the group of points of an elliptic curve, therefore we will be working with the additive notation for the group operation: we will write $[x]P$ for the group element obtained by adding P to itself x times. The group description $(\mathbb{G}, q, P)$ is public. A secret key is an element $x \in \mathbb{Z}_q$, and the corresponding public key is $X = [x]P \in \mathbb{G}$.

We begin with an interactive protocol – *Schnorr's Identification Scheme* – where a prover $\mathcal{P}$ holding the secret key x wants to convince a verifier $\mathcal{V}$, who only holds the public key X, that it knows the discrete logarithm of X to the base P, without revealing anything else about x. We will later apply a transformation to turn this protocol into a signature scheme. The protocol consists of the following three-move interaction:

Prover $\mathcal{P}(P, X, x)$		**Verifier** $\mathcal{V}(P, X)$
$k \leftarrow\$ \mathbb{Z}_q$		
$R \leftarrow [k]P$		
	R (commitment) $\longrightarrow$	
		$t \leftarrow\$ \mathbb{Z}_q$
	$\longleftarrow$ t (challenge)	
$s \leftarrow k - xt \pmod q$		
	s (response) $\longrightarrow$	
		accept iff $[s]P + [t]X = R$

(a) Show that this protocol is complete, i.e. that an honest prover always convinces the verifier.

Solution. An honest prover computes $s = k - xt \pmod{q}$, so the verifier checks

$$[s]P + [t]X = [k - xt]P + [t]([x]P) = [k - xt + xt]P = [k]P = R\,,$$

and accepts.

(b) Show that if the prover can predict the verifier's challenge t before sending the commitment R, then it can compute a valid response s without knowing the secret key x.

Solution. If the prover can predict t, it can choose $s \leftarrow\!\!\$\ \mathbb{Z}_q$ and compute $R = [s]P + [t]X$. Then, it sends R to the verifier, who will send back the challenge t. The prover can then send s as the response, and the verifier will accept since $[s]P+[t]X = R$ by construction.

(c) One can show that this protocol leaks nothing about x to a verifier that follows the protocol honestly and that a cheating prover that does not know x can only convince the verifier with probability $1/q$. We will not prove these properties here, but they are crucial for the security of the signature scheme we will derive from this protocol. What we need to convert this protocol into a signature scheme is to make it *non-interactive*. To this end, we use the *Fiat-Shamir transform*. Assume we want to sign a message m, then we replace the verifier's random challenge t with $H(R \,\|\, m)$. This allows the signer to honestly generate a challenge without interacting with the verifier. The resulting signature scheme is given below.

Schnorr Signature Scheme. Let $\mathbb{G}$ be a cyclic group of prime order q with generator P, as above, and let $H : \{0,1\}^* \to \mathbb{Z}_q$ be a hash function. The Schnorr signature scheme is defined as follows:

KeyGen	Choose a random $x \in \mathbb{Z}_q$ and compute $X = [x]P$. The public key is $pk = X$ and the secret key is $sk = x$.
Sign	To sign a message $m \in \{0,1\}^*$, choose a random nonce $k \in \mathbb{Z}_q$ and compute $R = [k]P$. Compute $t = H(R \,\|\, m)$ and $s = k - xt \pmod{q}$. The signature is $\sigma = (s, t)$.
Verify	To verify a signature $\sigma = (s, t)$ on a message m, compute $R' = [s]P + [t]X$ and $t' = H(R' \,\|\, m)$. The signature is valid if and only if $t' = t$.

Informally argue why this makes the scheme non-interactive, while preserving the security properties of the original protocol.

Solution. The challenge must be something that the signer cannot predict before choosing the commitment R. By using a hash function, we can ensure that the prover must commit to R before knowing the challenge. Adding the message m ensures that the signature is bound to the message. To formally prove security one must work in the *Random Oracle Model* (ROM), where the hash function acts as a random function that every party can query. This means also that the challenge is essentially random, preserving the security properties of the original protocol.

(d) Show that, if an adversary knows the value of the nonce k used to sign a message m, then it can compute the secret key x (except with negligible probability).

Solution. If the adversary knows k, then, since it also knows s and t from the signature $\sigma = (s, t)$, it can compute $x = (k - s)t^{-1} \pmod{q}$. The intuition is that the signature equation is a linear equation with only one unknown, x, and thus can be solved easily. The negligible probability comes from the fact that t could be zero, in which case the adversary cannot compute x.

(e) Show that, if the signer uses the same nonce k to sign two different messages m_1 and m_2, then an adversary can compute the secret key x (assuming no hash collisions happen).

Solution. Let $\sigma_1 = (s_1, t_1)$ and $\sigma_2 = (s_2, t_2)$ be the signatures on messages m_1 and m_2, respectively. Then, we have

$$s_1 = k - xt_1 \pmod{q}$$
$$s_2 = k - xt_2 \pmod{q}$$

Subtracting these two equations gives us

$$s_1 - s_2 = x(t_2 - t_1) \pmod{q}$$

Since $t_1 \neq t_2$ (because the messages are different), we can solve for x:

$$x = (s_1 - s_2)(t_2 - t_1)^{-1} \pmod{q}$$

The intuition is that, even though the adversary does not know k, two signatures give two linear equations with two unknowns, k and x, which can be solved easily.

Problem 2. We now show that if the most significant bits of the nonce k are known to an adversary, then it can compute the secret key x after seeing enough signatures. For simplicity, we will analyze the case where we assume that the L most significant bits of k are zero.[1] Let $N = \lceil \log_2 q \rceil$ be the bit-length of q. Then, we can write

$$k = 2^{N-L-1} + \varepsilon$$

where ε is a small error term, i.e. $|\varepsilon| \leq 2^{N-L-1}$. Next consider the signature equation:

$$s = k - xt \pmod{q}$$

which can be rewritten as

$$xt = k - s = 2^{N-L-1} - s + \varepsilon \pmod{q}.$$

Set $u = 2^{N-L-1} - s$ to obtain:

$$xt = u + \varepsilon \pmod{q}$$

with $|\varepsilon|$ small and t and u known. In other words, we have that u is a "good" approximation of xt modulo q. The problem of finding x given many such approximations is known as the *Hidden Number Problem* (HNP). That is, given n pairs (t_i, u_i) such that $xt_i = u_i + \varepsilon_i \pmod{q}$ with $|\varepsilon_i|$ small, find x.

Our strategy will be to reduce the HNP to an instance of the Closest Vector Problem (CVP) in a lattice, where the target vector is composed of publicly known values and the closest vector contains the secret key x in its last coordinate.

[1] One can easily extend to the case where the MSBs are known but not necessarily zero.

(a) Consider the following basis matrix B for a lattice $\mathcal{L}_B$:

$$B = \begin{bmatrix} q & 0 & 0 & \cdots & 0 & 0 \\ 0 & q & 0 & \cdots & 0 & 0 \\ 0 & 0 & q & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & q & 0 \\ t_1 & t_2 & t_3 & \cdots & t_n & 1/2^{L+1} \end{bmatrix}$$

and let $\underline{u} = (u_1, u_2, \ldots, u_n, 0)$. Prove that there exists a vector $\underline{v}$ in $\mathcal{L}_B$ such that:

$$\|\underline{u} - \underline{v}\| < \sqrt{n+1} \cdot 2^{N-L-1}.$$

Hint: Let $\underline{v} = (-\ell_1, -\ell_2, \ldots, -\ell_n, x) \cdot B$, where each ℓ_i is the integer such that $xt_i = u_i + \varepsilon_i + \ell_i q$ (over $\mathbb{Z}$).

Solution. Let $\underline{v} = (-\ell_1, -\ell_2, \ldots, -\ell_n, x) \cdot B = (u_1 + \varepsilon_1, u_2 + \varepsilon_2, \ldots, u_n + \varepsilon_n, x/2^{L+1})$. Then, we have

$$\begin{aligned} \underline{u} - \underline{v} &= (u_1, u_2, \ldots, u_n, 0) - (u_1 + \varepsilon_1, u_2 + \varepsilon_2, \ldots, u_n + \varepsilon_n, x/2^{L+1}) \\ &= (-\varepsilon_1, -\varepsilon_2, \ldots, -\varepsilon_n, -x/2^{L+1}), \end{aligned}$$

and thus

$$\|\underline{u} - \underline{v}\|^2 = \sum_{i=1}^{n} \varepsilon_i^2 + (x/2^{L+1})^2 < n \cdot 2^{2(N-L-1)} + 2^{2(N-L-1)} = (n+1)2^{2(N-L-1)},$$

which implies that

$$\|\underline{u} - \underline{v}\| < \sqrt{n+1} \cdot 2^{N-L-1}.$$

(b) Recall the Gaussian heuristic from the lecture: this states that, for a "random" lattice $\mathcal{L}$ in n' dimensions, we have:

$$\lambda_1 \approx \sqrt{\frac{n'}{2\pi e}} \cdot \det(\mathcal{L})^{1/n'}.$$

Compute the determinant of B and the expected value of λ_1 for the corresponding lattice (under the Gaussian heuristic).

Solution. The matrix is lower triangular, so computing its determinant is just a matter of multiplying all elements in the diagonal together.

$$\det(\mathcal{L}_B) = q^n / 2^{L+1}$$

Then, the expected value of λ_1 will be:

$$\lambda_1(\mathcal{L}_B) \approx \sqrt{\frac{n+1}{2\pi e}} \det(\mathcal{L}_B)^{1/(n+1)} = \sqrt{\frac{n+1}{2\pi e}} \frac{q^{n/(n+1)}}{2^{(L+1)/(n+1)}}$$

(c) Compare the distance $\|\underline{u} - \underline{v}\|$ from Part 2.a to the expected value of λ_1 from Part 2.b, showing that $\underline{u}$ and $\underline{v}$ are abnormally close. Then, looking at the last coordinate of $\underline{v}$, explain how to recover the secret key x assuming we have an algorithm that solves the CVP for the lattice $\mathcal{L}_B$.

Hint: Check that for large enough n, $\|\underline{u} - \underline{v}\| < \lambda_1(\mathcal{L}_B)/2$. It might be helpful to use the approximations $x < q \approx 2^N$, $\sqrt{2\pi e} \approx 2^{2.05}$ and work by taking logarithms.

Solution. We have

$$\|\underline{u} - \underline{v}\| < \sqrt{n+1} \cdot 2^{N-L-1}$$

and

$$\begin{aligned}
\lambda_1(\mathcal{L}_B) &\approx \sqrt{\frac{n+1}{2\pi e}} \frac{q^{n/(n+1)}}{2^{(L+1)/(n+1)}} \\
&\approx \sqrt{\frac{n+1}{2\pi e}} \cdot 2^{Nn/(n+1)} / 2^{(L+1)/(n+1)} \\
&= \sqrt{\frac{n+1}{2\pi e}} \cdot 2^{(Nn-L-1)/(n+1)}.
\end{aligned}$$

So comparing them gives:

$$\begin{aligned}
N - L - 1 &< (Nn - L - 1)/(n+1) - 1 - 2.05 \\
(N - L + 2.05)(n+1) &< Nn - L - 1 \\
n(L - 2.05) &> N + 3.05
\end{aligned}$$

This equation also tells us precisely how many signatures we need to see in order to recover the secret key x. Plugging in $N = 256$ and $L = 8$, we get $n > (256+3.05)/(8-2.05) \approx 43.5$, so we need to see at least 44 signatures to recover the secret key x. As a side note, in this case the information-theoretical minimum number of signatures needed to recover x is $N/L = 32$.

Note, however, that we are comparing $\|\underline{u} - \underline{v}\|$ with an *estimate* of $\lambda_1(\mathcal{L}_B)$, which only holds for random lattices and large n. In practice, we may need to see more signatures to recover x.

To conclude, if we have an algorithm that solves the CVP for the lattice $\mathcal{L}_B$, we can hope to find the vector $\underline{v}$ that is closest to $\underline{u}$. If the recovery retrieves the intended $\underline{v}$, its last coordinate is $x/2^{L+1}$, from which we can recover the secret key x.

(d) Recall that, in the Kannan embedding approach to solving CVP, we augment the lattice B to obtain:

$$B' = \begin{bmatrix} B & 0 \\ \underline{u} & M \end{bmatrix}.$$

Here $\underline{u}$ is the target input vector for CVP. Let $\underline{v}$ be the solution to the CVP problem. A shortest vector in the lattice $\mathcal{L}' = \mathcal{L}_{B'}$ is expected to be of the form $(\underline{f}, M)$ where $\underline{f} = \underline{u} - \underline{v}$. Compute an upper bound on the norm of $(\underline{f}, M) \in \mathcal{L}'$ (as a function of M).

Solution. Observe that $\|(\underline{f}, M)\|^2 = \|\underline{f}\|^2 + \|M\|^2 = \|\underline{f}\|^2 + M^2$. We have already computed $\|\underline{f}\|$ in Part 2.a:

$$\|\underline{f}\| < \sqrt{n+1} \cdot 2^{N-L-1}$$

Thus:

$$\|(\underline{f}, M)\| < \sqrt{(n+1) \cdot 2^{2(N-L-1)} + M^2}$$

(e) Compute the determinant of B' and the expected length of the shortest vector λ_1' for the corresponding lattice (under the Gaussian heuristic).

Solution. The matrix is lower triangular, so computing its determinant is just a matter of multiplying all elements in the diagonal together. We have $\det(\mathcal{L}_{B'}) = \det(\mathcal{L}_B) \cdot M = q^n M / 2^{L+1}$.

Then, the expected value of λ_1' will be:

$$\lambda_1'(\mathcal{L}_{B'}) \approx \sqrt{\frac{n+2}{2\pi e}} \det(\mathcal{L}_{B'})^{1/(n+2)} = \sqrt{\frac{n+2}{2\pi e}} \frac{q^{n/(n+2)} M^{1/(n+2)}}{2^{(L+1)/(n+2)}}$$

(f) Compare the estimate of λ_1' from Part 2.e to the bound on $\|(\underline{f}, M)\|$ you obtained in Part 2.d. What does this suggest about how to set M in order to ensure the success of the Kannan embedding approach to solving CVP in this case?

Solution. This is a tricky question! At face value, the lemma from the lectures tells us to set $M = \|\underline{f}\|$. The problem with this is that we only have a (possibly loose) upper bound on $\|\underline{f}\|$. On top of that, our objective is to set M such that our vector $(\underline{f}, M)$ is the shortest vector in the lattice, so significantly smaller than λ_1'. However, our estimate of λ_1' (from applying the Gaussian heuristic) might be imprecise: the Gaussian heuristic usually starts working consistently for large n ($n \geq 45$) and for random lattices. Our lattices are often small and not-so-random (see the next task for an example).

As a general approach, set up the inequality $\|\underline{f}, M\| < \lambda_1'/2$ and see what it tells you about M. You will find that the inequality is not very clean, but you can try to work it out numerically with some values of M. In other words, make educated guesses about what the value of M should be and then experiment around until it works consistently. This is somewhat common practice in cryptanalysis: let the theory guide you, but do not let it constrain you! Sometimes you don't have the instruments to fulfill all the conditions that theory requires, but more often than not, cryptanalytic tools work better than the theory predicts them to do (see the provable bounds on LLL, and then compare to the actual performance of LLL in practice).

Also remember that Kannan embedding is pretty forgiving: there are many values that work. You may use any value that you believe works consistently across instances of the problem.

(g) Consider $\underline{z} = (-t_1, -t_2, -t_3, \ldots, -t_n, q, 0) \cdot B' \in \mathcal{L}'$. Compute the norm of $\underline{z}$ and compare it to the bound on the norm of $(\underline{f}, M)$ you obtained in Part 2.d, showing that it is typically about $\sqrt{n+1}$ times smaller (irrespective of the choice of M).

Solution. We have:

$$\begin{aligned}\underline{z} &= (-t_1, ..., -t_n, q, 0) \cdot B' \\ &= (-t_1 \cdot q + q \cdot t_1, ..., -t_n \cdot q + q \cdot t_n, q/2^{L+1}, 0) \\ &= (0, ..., 0, q/2^{L+1}, 0)\end{aligned}$$

This vector has norm $q/2^{L+1}$. Using the same approximation as in Part 2.a ($x < q < 2^N$), this yields an upper bound for the squared norm of $\underline{z}$:

$$\|\underline{z}\|^2 < 2^{2(N-L-1)}$$

Comparing this with the bound on the squared norm of $(\underline{f}, M)$ from Part 2.d, one can see that the bound is at least $n + 1$ times smaller, if we ignore the value of M. By applying the square root, the result follows.

The existence of this vector $\underline{z}$ also suggests that you may not always recover the secret key x from the solution to the CVP problem. In fact, the vectors $(\underline{f}, M) \pm \underline{z}$ are also in the lattice, but you may still be able to recover x from either of them...

This problem shows that $\mathcal{L}'$ contains unusually short vectors, so solving SVP may not directly yield a solution to the original CVP for the particular lattices we have constructed here.

On the other hand, perhaps the required solution will appear *elsewhere* in the basis obtained from performing lattice reduction on B'...